

## PRÁCTICA: Unidad 6 - Análisis Morfológico

### Ejercicio 1: Identificación de Patrones de un Cubo Rubik

A partir del *dataset* de imágenes que se muestra en la figura 1 (archivos *rubik\_<id>.jpg*), se requiere realizar un **análisis automático** mediante técnicas de PDI, sobre cada cara de un cubo Rubik.

El objetivo principal es **detectar e identificar el color y la ubicación** de cada elemento sobre cada una de las caras, para que los resultados sean los parámetros de entrada de algún algoritmo de resolución.

Para ello, considere cada una de las imágenes del *dataset* provisto y desarrolle un *script* que itere sobre las mismas y de forma automática, permita:

1. Generar una imagen final con una etiqueta (texto) sobre cada elemento, acusando su respectiva posición y color.
2. Generar una matriz de tamaño 9x9 cuyos elementos corresponden al color de cada elemento en su posición.

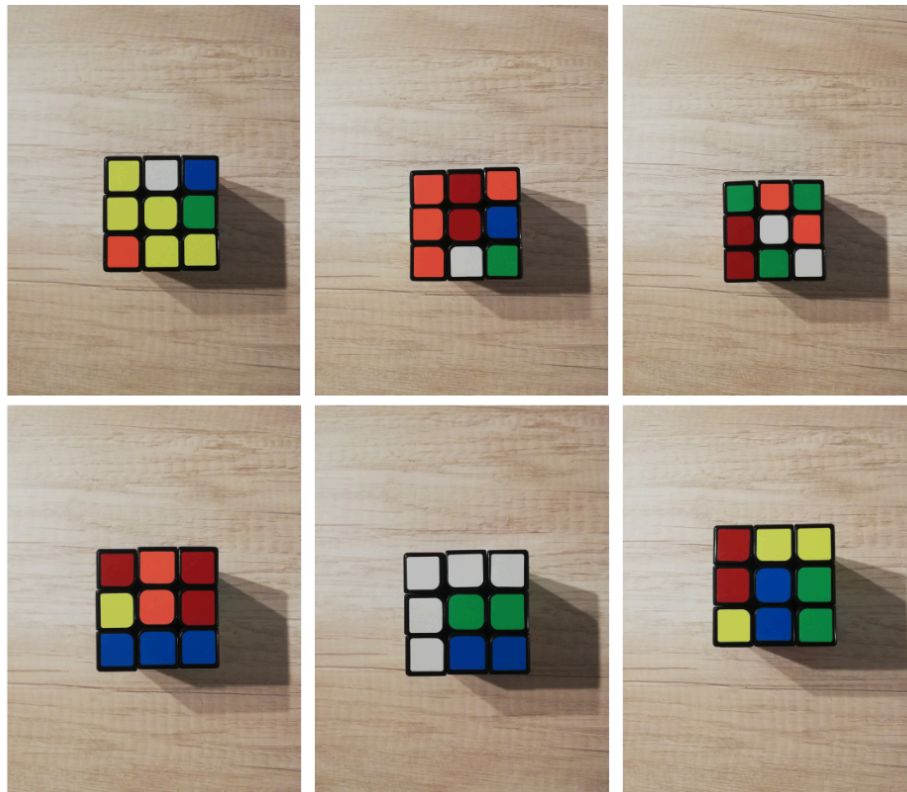


Figura 1: *Dataset* de imágenes de las caras de un cubo Rubik.

## **Ejercicio 2: Generación de Puntos sobre una PCB**

A partir del *dataset* de imágenes que se muestra en la figura 2 (archivos *pcb\_<id>.jpeg*), se requiere realizar un **análisis automático** mediante técnicas de PDI, sobre cada placa de circuito impreso (PCB).

El objetivo principal es **generar una serie de puntos únicamente sobre el área de la PCB** que se encuentra en cada imagen. Estos puntos se deben distribuir de forma equidistante con un salto en milímetros preestablecido (mediante el uso del código ArUco), considerando que los resultados obtenidos serán los parámetros de entrada de algún algoritmo de generación de G-Code (lenguaje utilizado para comandar un sistema CNC).

Para ello, considere cada una de las imágenes del *dataset* provisto y desarrolle un *script* que itere sobre las mismas y de forma automática, permita:

1. Generar una imagen final con los puntos impresos sobre la PCB detectada.
2. Calcular la cantidad total de puntos impresos y elaborar un diccionario con el formato: `{'punto_1': (<coordenada_x>, <coordenada_y>), ... }`

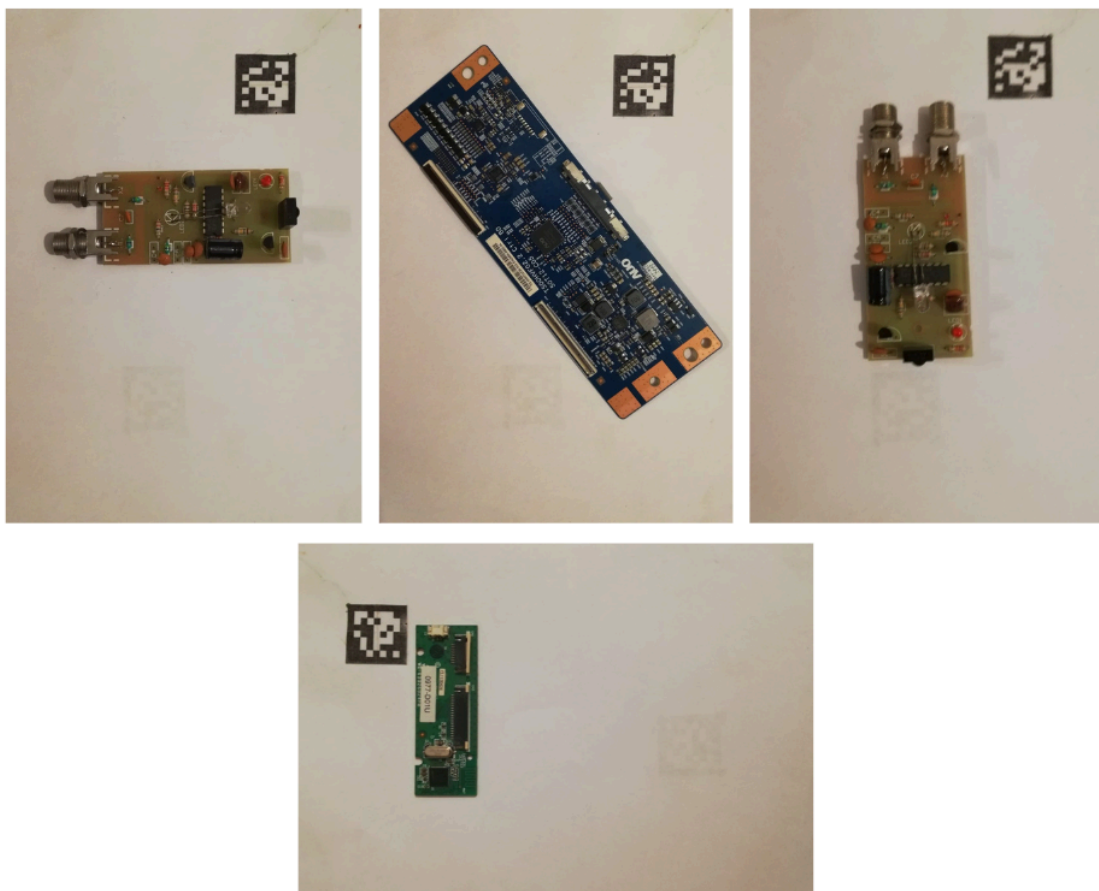


Figura 2: *Dataset* de imágenes de un conjunto de PCB.



**AYUDA:** Para aproximar un factor de conversión de milímetros a píxeles se debe considerar que el código ArUco ([https://docs.opencv.org/4.x/d4/d13/tutorial\\_aruco\\_detection.html](https://docs.opencv.org/4.x/d4/d13/tutorial_aruco_detection.html)) mide 23x23 milímetros y se deben obtener la ubicación de sus vértices con el uso de las funciones:

- `aruco_dict=cv2.aruco.getPredefinedDictionary(dict=cv2.aruco.DICT_7X7_50)`
- `corners,_,_=cv2.aruco.detectMarkers(image=input_image,dictionary=aruco_dict,parameters=cv2.aruco.DetectorParameters())`

Con estos datos, se puede hallar una aproximación del factor de conversión mediante la operación:

$$factor_{mm \rightarrow px} = \frac{S}{\|\overline{p_0} - \overline{p_1}\|}$$

Es decir,

$$factor_{mm \rightarrow px} = \frac{\text{Lado del ArUco en milímetros}}{\text{Distancia en píxeles entre dos esquinas consecutivas del ArUco}}$$